Implementing GitHub Copilot in Engineering Teams

Shared setup, workflow, and guardrails

github copilot · copilot cli · team workflow

SHARED WAY OF WORKING

Why this needs a shared AI operating model

The problem is not access. The problem is using Copilot with one shared standard.

- AI is already part of development work.
- Many teams already use Copilot, but not always in the same way.
- Without one shared model, AI adoption creates drift, review noise, and uneven quality.

```
BACKEND / AUTH / REQUEST-EMAIL-CODE.TS     AUTH FLOW

12   const normalizedEmail = email.trim()
13   if (!isValidEmail(normalizedEmail)) return…
14   await authService.requestCode(normalizedEm…
15   return 'ok'
16   const invalidEmailError = buildInva…   tab
17   if (invalidEmailError) return invalidEmail…
18   // keep email validation shared across aut…

$ copilot /plan "extract invalid email validati…
```

PRACTICAL MODEL

How Copilot works in practice

- Copilot is not only autocomplete.
- It works through context, instructions, policies, and workflow.
- The result depends on the environment we create around it.

LAYERED MODEL

Three control layers

Rollout only works when these three layers reinforce each other.

Enterprise / organization

policy and access

AI controls -> Copilot -> CLI

Repository

shared rules

.github/copilot-instructions.md

Local tool and user environment

execution context

$HOME/.copilot/copilot-instructions.md

What Enterprise and the organization control

Access

seat assignment and org-level delegation

AI CONTROLS

Policies & models

feature, privacy, and model controls

POLICIES / MODELS

ENTERPRISE / ORGANIZATION

AI Controls

GitHub-native AI surfaces

cloud agent, code review, MCP

AGENTS / MCP

Visibility

metrics dashboards and APIs

DASHBOARD / API

What an AI repo setup looks like

PARENT WORKSPACE

* OPTIONAL

frontend

.github/copilot-instructions.md

REPOSITORY BASELINE

AGENTS.md

COMPLEMENTARY

backend

.github/copilot-instructions.md

REPOSITORY BASELINE

AGENTS.md

COMPLEMENTARY

REPOSITORY LAYER

What each repository must define

Shared rules for the code it owns

tech stack, architecture, coding guidelines with reasoning

```
.github/copilot-instructions.md
```

Contract ownership with the other side

path-specific rules with `applyTo` glob patterns per area

```
.github/instructions/*.instructions.md
```

Real verification commands

commands in the file, not just tool names

```
npm test · pytest -v · npm run build
```

Bad prompts vs good prompts

✗ BAD PROMPTING

- Vague ask
  "Fix the API"
- Too much scope
  "Rewrite the whole auth flow"
- No success criteria
  No mention of what done looks like
- No verification
  No test or check requested
- No ownership boundary
  Touches frontend and backend at once

✓ GOOD PROMPTING

- Clear goal
  "Add projectId validation to POST /projects"
- Relevant context
  "Route is in routes/projects.routes.js"
- Constraints
  "Do not change the response shape"
- Expected output
  "Return 400 if projectId is missing"
- One small next step
  "Only the validation — not the DB query"

The Copilot CLI patterns

`conversation`	scoped questions, fast context, no task switch
`/plan`	let Copilot inspect before coding, for complex or risky work
`/review`	ask for gaps, risks, and what was not verified
`skills`	repeatable specialist tasks without rewriting the prompt
`agents / subagents`	specialized roles and parallel delegation when complexity justifies it

LIVE DEMO 01

Add a loading state to the login button

DIRECT PROMPT

"Add a loading state to the login button"

- Copilot infers from the open file
- No codebase inspection
- May miss existing patterns

/PLAN

/plan "Add a loading state to the login button"

- Inspects auth hook, button component, existing patterns
- Asks clarifying questions before writing
- Generates a plan: review, approve, execute one step

Let's try it.

Skills and custom agents

SKILLS

`/skills list`

see what is available

`/generate-tests "..."`

invoke by name

repeatable instruction packages, define once, invoke anywhere

`.github/skills/SKILL.md`

AGENTS

`/agent "..."`

open the agent selector

`Use the security-auditor agent to check ...`

explicit invocation

own tools, model, and behavioral scope

`.github/agents/<name>.agent.md`

Both live in the repository, the team shares them.

Subagents

/fleet "..."

parallel subtasks, each agent runs in its own isolated context

/delegate "..."

async cloud handoff, commits to a new branch, opens a draft PR

Premium request cost scales with parallelism, use when the task justifies it.

LIVE DEMO 02

Agents and fleet

> `.github/agents/test-writer.agent.md`
>
> writes Jest + Supertest tests for a route

> `.github/agents/sql-reviewer.agent.md`
>
> reviews SQL in model files for injection risks

Let's fleet them.

Thank you.

Any questions?

Desde la Vega Baja con ❤️ José Miguel Torres Hernández